

Laboratório de Experimentação de Software — Relatório Final

Lab01 — Repositórios Populares + Setup do Kanban

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab01 — Repositórios Populares
Grupo (trio)	Matheus Ferreira Tilton; Murilo Andrade Machado; Pedro Henrique Barbosa Montandon de Araújo
Link do repositório / GitHub Projects	https://github.com/matheustilton/lab01-repositorios-populares
Data de entrega	27 de Agosto de 2026

1. Introdução

O GitHub concentra milhões de repositórios, mas apenas uma fração pequena acumula popularidade massiva, medida em número de estrelas. Entender que características esses repositórios têm em comum — idade, atividade, forma de contribuição, linguagem — ajuda a diferenciar o que é sinal de maturidade e qualidade de engenharia do que é apenas efeito de rede (um projeto conhecido atrai mais atenção, independente de seus atributos técnicos). Este laboratório investiga essa questão de forma empírica, minerando os 1.000 repositórios mais estrelados do GitHub via API GraphQL.

O enunciado define as seguintes Questões de Pesquisa:

- RQ01 — Sistemas populares são maduros/antigos?
- RQ02 — Sistemas populares recebem muita contribuição externa?
- RQ03 — Sistemas populares lançam releases com frequência?
- RQ04 — Sistemas populares são atualizados com frequência?
- RQ05 — Sistemas populares são escritos nas linguagens mais populares?
- RQ06 — Sistemas populares possuem alto percentual de issues fechadas?

Hipóteses informais (registradas após a coleta piloto de 100 repositórios e antes da coleta completa de 1000 — ver docs/hipoteses.md):

RQ	Hipótese informal
RQ01	Sim, com mediana bem acima de cinco anos — estrela é contador que só cresce, e acumular centenas de milhares exige tempo de exposição.
RQ02	Sim em volume absoluto, mas com distribuição muito assimétrica — o número de PRs mede mais o modelo de governança do projeto do que popularidade em si.
RQ03	Não — mediana baixa e fração grande com zero releases, porque boa parte dos repositórios mais estrelados não é software instalável (listas, roteiros de estudo).
RQ04	Sim, fortemente, com mediana de poucos dias — popularidade atrai contribuição contínua, mesmo em projetos com núcleo estável.

RQ	Hipótese informal
RQ05	Parcialmente — concordância no topo do ranking com divergência na ordem, porque o Octoverse mede contribuidores e este estudo mede estrelas.
RQ06	Sim, com mediana acima de 0,8 — projetos populares têm triagem ativa de issues.
RQ07	Não — a linguagem explica pouco; o tipo de projeto explica mais. Se houver padrão, deve aparecer mais em releases do que em PRs ou atualidade.

Contribuições próprias do grupo além do enunciado (30% de inovação — detalhadas na seção 3.6):

- RQ07: RQ02, RQ03 e RQ04 desdobradas por linguagem primária, para investigar se a linguagem por si só explica diferenças de contribuição, releases e atualidade.
- Validação cruzada da coleta contra a API REST do GitHub, uma fonte independente da GraphQL usada na mineração.
- Auditoria automática do board a cada snapshot, sinalizando cartões sem Assignee, sem Issue real ou com Status inconsistente com o estado da Issue.

2. Contexto

Este é o Lab01 da disciplina, o primeiro de uma sequência de cinco laboratórios. Além de responder às RQs sobre repositórios populares, ele estabelece o GitHub Projects (Kanban) que acompanhará o grupo até o Lab05 — os snapshots exportados a cada sprint (seção 3.3) formam a base de dados que os laboratórios 4 e 5 vão consumir.

O objeto de estudo são os 1.000 repositórios com maior número de estrelas do GitHub, obtidos via GraphQL API v4 (query search(type: REPOSITORY, query: "stars:>1 sort:stars-desc")). Para a RQ05 e a RQ07, a referência de "linguagens mais populares" adotada é o GitHub Octoverse 2025 (<https://octoverse.github.com/>), mantida como única fonte do início ao fim do laboratório, conforme docs/fontes.md.

Ressalva metodológica importante: o Octoverse ordena linguagens por número de contribuidores, enquanto este estudo ordena repositórios por estrelas. São medidas diferentes de popularidade — contribuidores medem quem escreve código, estrelas medem quem marca o projeto como relevante. Essa divergência é justamente o que a RQ05 investiga, não uma limitação da fonte escolhida.

3. Metodologia

3.1 Principais Desafios

- Limite de taxa (rate limit) da API GraphQL: com páginas de 25 repositórios, a API retornava erro 502 de forma determinística ao redor do rank 76–100, porque cada repositório pede quatro totalCount agregados e o custo por requisição estourava o tempo de resposta. A página foi reduzida para 10 repositórios, estável em toda a coleta de 1000.
- Paginação de 1.000 repositórios: a busca do GitHub devolve no máximo 1.000 resultados por consulta — os 1.000 pedidos cabem exatamente nesse teto, sem margem para filtro adicional depois.
- Contagem de releases censurada no teto de 1.000: a conexão releases da GraphQL para de contar em 1.000. Repositórios que reportam exatamente esse valor têm contagem real maior (ex.: ggml-org/llama.cpp aparece com 1000 quando o valor real é 6.844). Afeta 21 dos 1.000 repositórios (2,1%).

- Ausência de histórico de mudança de status no GitHub Projects: a API não expõe quando um cartão mudou de coluna, o que exigiu snapshots manuais recorrentes a cada fechamento de sprint (data/snapshots_board.csv).

3.2 Tomadas de Decisão

- Limite de WIP na coluna Doing: 3, um por integrante do trio — evita que alguém abra frentes paralelas e acumule trabalho pela metade, e força quem termina antes a revisar o que está em Review em vez de puxar tarefa nova (ver Issue #5).
- pushedAt em vez de updatedAt para a RQ04: updatedAt muda com eventos que não são alteração de código (estrela recebida, edição de descrição), o que infla artificialmente a atualidade aparente dos repositórios mais populares.
- PRs MERGED, não CLOSED, para a RQ02: contribuição aceita é PR incorporada; PR fechado sem merge não é contribuição.
- Mediana, não média, em todas as RQs numéricas: as distribuições são fortemente assimétricas (ex.: a média de PRs aceitos é 5,5× a mediana) — poucos outliers gigantes dominariam a média.
- Repositórios sem linguagem primária: rotulados Undefined, tratados como categoria própria (listas de links, material de estudo), não como dado faltante.
- Repositórios sem nenhuma issue (RQ06): razão definida como 0,0 por convenção, mas excluídos do cálculo principal da mediana — "não teve issue" não é o mesmo que "não resolveu issue", e incluí-los puxaria o resultado para baixo de forma artificial.
- Linguagens com poucos repositórios (RQ07): agrupadas em "Outras" quando $n < 30$, limiar já usado na validação de consistência da Sprint 2 — mediana sobre 1 ou 2 repositórios não sustenta comparação.

3.3 Etapas

Sprint	Entregas	Responsável(is)	Issues (nº)
Lab01S01	Consulta GraphQL de 100 repositórios; GitHub Projects criado com colunas e WIP definidos	Trio (setup do board)	#1, #5
Lab01S02	Paginação para 1.000 repositórios; validação de consistência (distribuição, outliers, ausentes) por RQ; hipóteses informais; snapshot Lab01S02	Murilo: RQ01+RQ02 · Matheus: RQ03+RQ04_RQ05 · Pedro: RQ06+RQ07	#12, #13, #14
Lab01S03	Análise estatística e visualização das 7 RQs	A: RQ01+RQ02 e infraestrutura de análise · B: RQ03+RQ04 · C: RQ05+RQ06+RQ07	#18, #19, #20
Relatório Final	Consolidação dos resultados, gráficos e discussão	Trio	#21, #22

A divisão acima reflete os Assignees reais das Issues no board (GitHub Projects), não apenas uma divisão narrativa — a correção do professor é feita a partir do board.

Configuração do processo

Colunas do board (campo Status): Backlog → Sprint Backlog → Doing → Review → Done — a segunda coluna, nomeada pelo grupo como "Sprint Backlog", cumpre o papel de "To Do" exigido pelo enunciado.

Limite de WIP na coluna Doing: 3 (justificativa na seção 3.2).

Regras acordadas: todo cartão é uma Issue real (sem draft issues); toda Issue tem Assignee antes de sair de Backlog; todo commit referência o número da Issue; cartões movidos apenas conforme o progresso real.

3.4 Ferramentas

- Python 3.11+, com script próprio de consulta à GraphQL API do GitHub — apenas `urllib.request` (biblioteca padrão); nenhuma biblioteca de terceiros consulta a API, conforme exigido pelo enunciado.
- pandas e Matplotlib, usados exclusivamente na etapa de análise (Lab01S03), que lê o CSV já coletado e não acessa a rede.
- pytest, para a suíte de testes do projeto (82 testes cobrindo mapeamento, paginação, métricas por RQ, amostragem e serialização).
- API REST do GitHub, como fonte independente para validação cruzada da amostra coletada via GraphQL.
- GitHub Projects (v2) como ferramenta de processo — <https://github.com/matheustitton/lab01-repositorios-populares>.

3.5 Tabela de Métricas

RQ	Métrica	Definição operacional	Unidade	Ferramenta / Fonte
RQ01	Idade do repositório	Data atual – createdAt, em anos	Anos	Script GraphQL próprio (campo createdAt)
RQ02	Contribuição externa	Total de pull requests com estado MERGED	PRs	Script GraphQL próprio (pullRequests(states: MERGED).totalCount)
RQ03	Frequência de releases	Total de releases publicadas (censurado em 1000 pela API)	Releases	Script GraphQL próprio (releases.totalCount)
RQ04	Frequência de atualização	Data atual – pushedAt, em dias	Dias	Script GraphQL próprio (campo pushedAt)
RQ05	Linguagem mais popular	primaryLanguage.name; Undefined quando ausente	Categórica	Script GraphQL próprio + GitHub Octoverse 2025 (referência)
RQ06	Percentual de issues fechadas	issues(CLOSED) / issues(total); 0,0 se total = 0	Razão [0,1]	Script GraphQL próprio (issues(states: CLOSED/OPEN))
RQ07 (bônus)	RQ02/RQ03/RQ04 por linguagem	Mediana de cada métrica, agrupada por primaryLanguage (n ≥ 30; resto agrupado em "Outras")	Múltiplas	src/analysis/rq_reports.py — rq07_by_language

3.6 Inovações Propostas pelo Grupo (30% da nota)

(a) Nova Questão de Pesquisa — RQ07 (bônus)

RQ02, RQ03 e RQ04 desdobradas por linguagem primária, para investigar se a linguagem de um repositório, isoladamente, explica diferenças de contribuição externa, frequência de releases e atualidade. Linguagens

com menos de 30 repositórios na amostra são agrupadas em "Outras", para não sustentar mediana sobre 1 ou 2 casos. O resultado aparece na seção 4.2 (gráfico RQ07) e é discutido na seção 4.3.

(c) Mudança de arquitetura — validação cruzada REST vs. GraphQL

Além da mineração via GraphQL, uma amostra determinística (priorizando casos de borda: repositório sem linguagem primária, sem releases, sem issues) é conferida contra a API REST do GitHub, uma fonte independente. Diferenças pequenas em stars e pushed_at entre as duas fontes são tratadas como deriva temporal esperada (o dataset é um retrato do instante da coleta); diferenças em created_at ou primary_language indicariam erro real de mapeamento — não houve nenhuma. Resultado completo em docs/validacao_amostra.md.

(c) Mudança de arquitetura — auditoria automática do board

O script de snapshot (src/cli/export_project_snapshot.py) audita o board a cada execução e avisa sobre três problemas que o próprio enunciado penaliza: cartões sem Assignee, cartões sem Issue real (draft) e cartões cujo Status contradiz o estado real da Issue. Isso transforma uma exigência de qualidade do processo (seção 3.3) em uma checagem automatizada, e não apenas em disciplina manual do grupo.

4. Resultados

4.1 Coleta de Dados

Os 1.000 repositórios-alvo foram coletados integralmente em 2026-08-18 (100 páginas de 10 repositórios, sem degradação de página), a um custo de 100 pontos de rate limit (de 5.000/hora disponíveis). Nenhum repositório foi descartado por dado incompleto — os casos de dado ausente ou limitado são tratados como categoria própria, não como remoção:

- RQ03: 21 repositórios (2,1%) com contagem de releases censurada no teto de 1.000 da API — mantidos no dataset, mas sinalizados: o valor real é maior e não deve ser usado para média/máximo.
- RQ05: 87 repositórios (8,7%) sem linguagem primária, rotulados Undefined.
- RQ06: 43 repositórios (4,3%) sem nenhuma issue (denominador zero) — excluídos apenas do cálculo da mediana principal, mantidos no dataset; a base interpretável da RQ06 é de 957 repositórios.

A validação de consistência completa (distribuição, outliers e valores ausentes por RQ) está documentada em docs/hipoteses.md e foi conferida, nesta análise final sobre os 1.000 repositórios, exatamente com os mesmos valores: mediana de idade 7,75 anos, mediana de PRs aceitas 768, mediana de releases 39, mediana de dias desde atualização 1,81, e mediana da razão de issues fechadas 0,8761 (excluindo os 43 repositórios sem issue).

4.2 Visualização Gráfica

Pergunta: RQ01 — Sistemas populares são maduros/antigos?

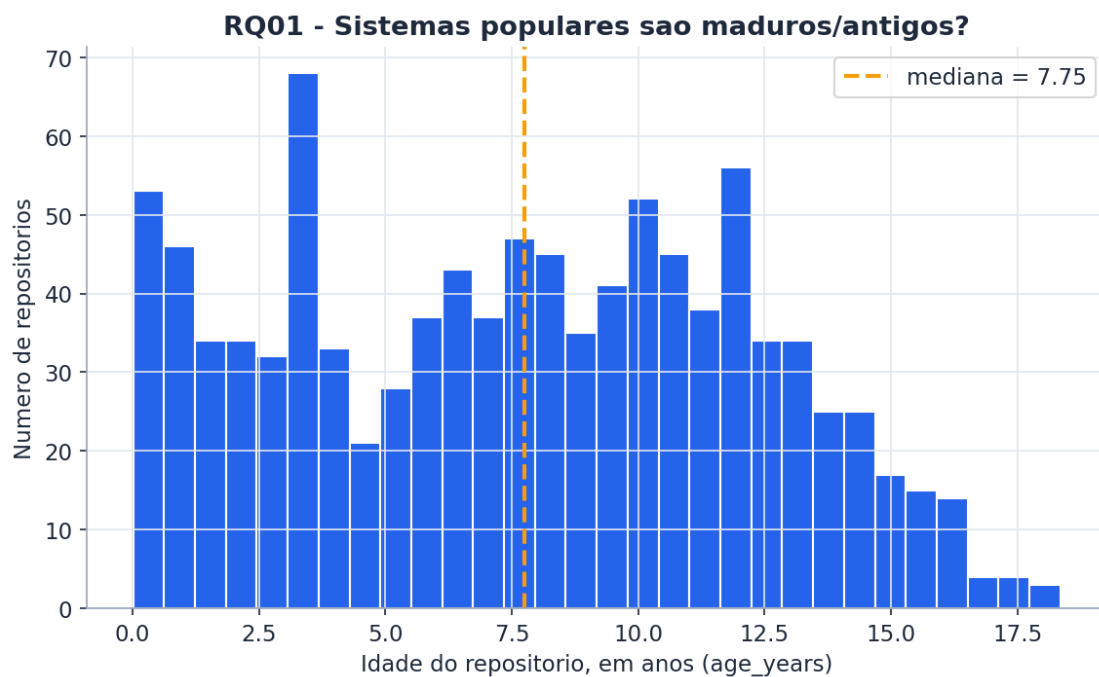


Figura 1 — Distribuição da idade dos repositórios (mediana = 7,75 anos).

Pergunta: RQ02 — Sistemas populares recebem muita contribuição externa?

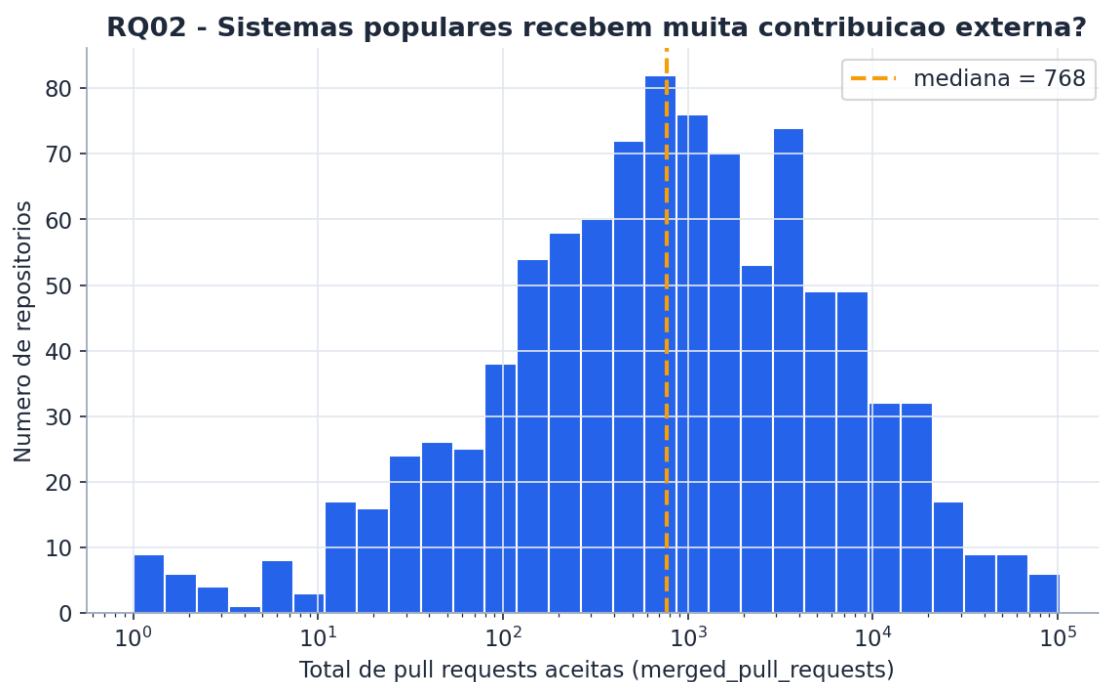


Figura 2 — Distribuição de PRs aceitas, escala logarítmica (mediana = 768).

Pergunta: RQ03 — Sistemas populares lançam releases com frequência?

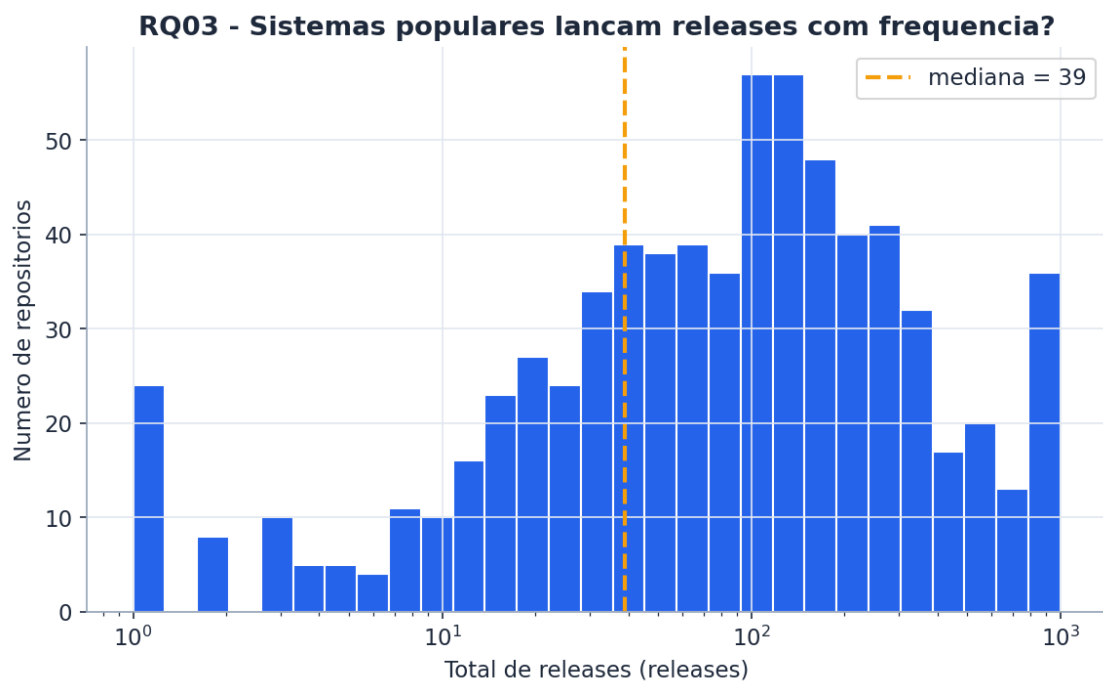


Figura 3 — Distribuição de releases, escala logarítmica (mediana = 39; 28,6% com zero releases).

Pergunta: RQ04 — Sistemas populares são atualizados com frequência?

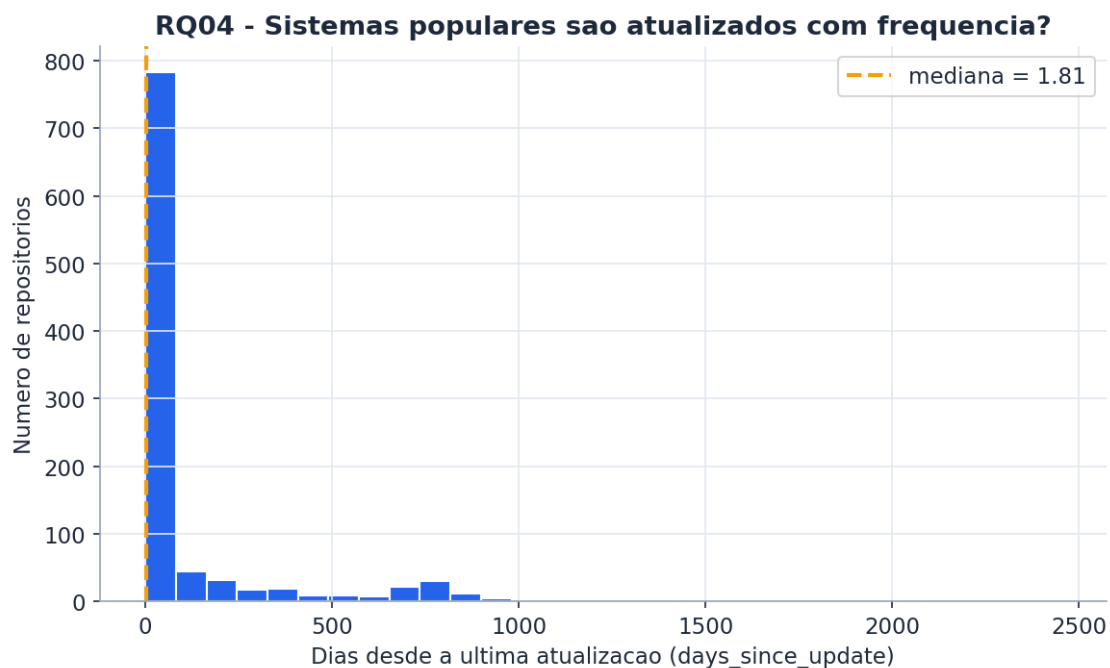


Figura 4 — Dias desde a última atualização (mediana = 1,81 dias).

Pergunta: RQ05 — Sistemas populares são escritos nas linguagens mais populares?

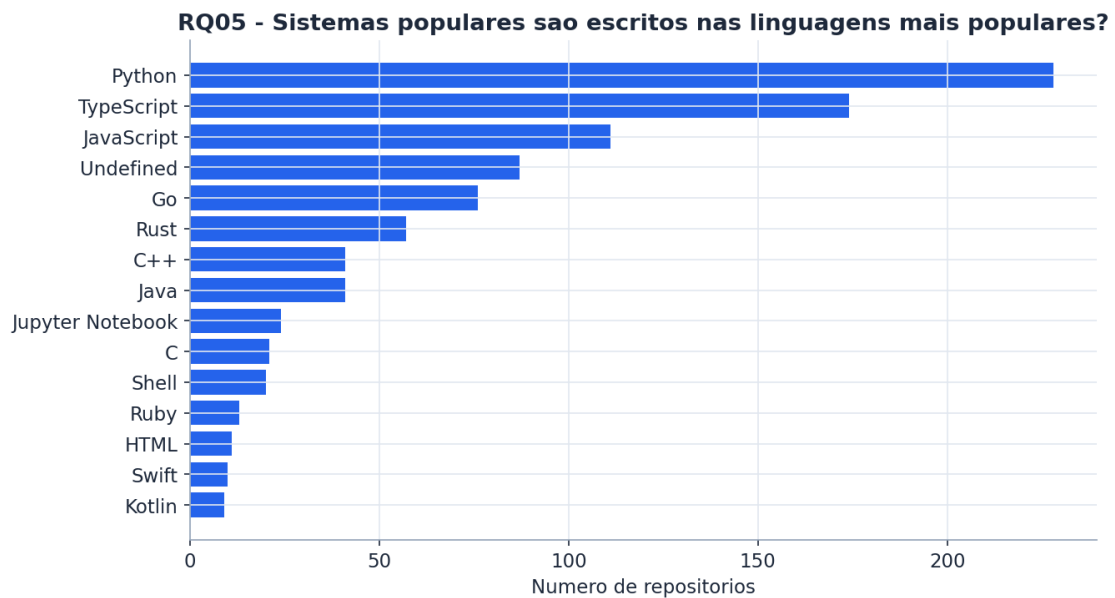


Figura 5 — Contagem de repositórios por linguagem primária (top 15 de 44 linguagens distintas).

Pergunta: RQ06 — Sistemas populares possuem alto percentual de issues fechadas?

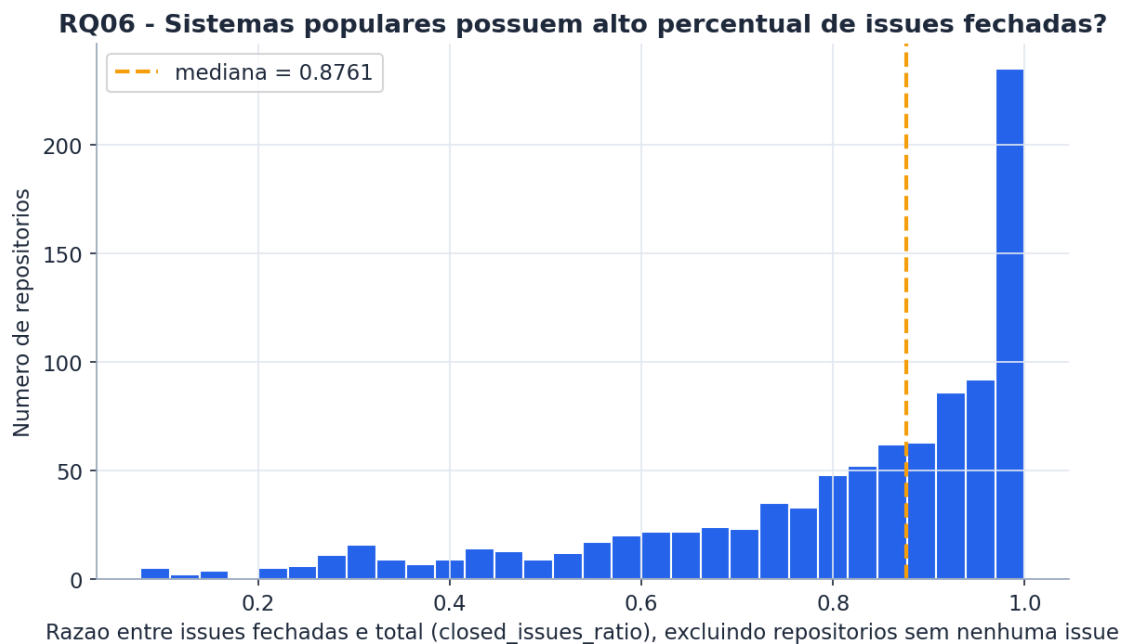


Figura 6 — Razão de issues fechadas, excluindo repositórios sem nenhuma issue (mediana = 0,8761).

Pergunta: RQ07 (bônus) — Linguagens populares recebem mais contribuição, lançam mais releases e são atualizadas com mais frequência?

RQ07 - Linguagens populares recebem mais contribuicao, lancam mais releases e sao atualizadas com mais frequencia?

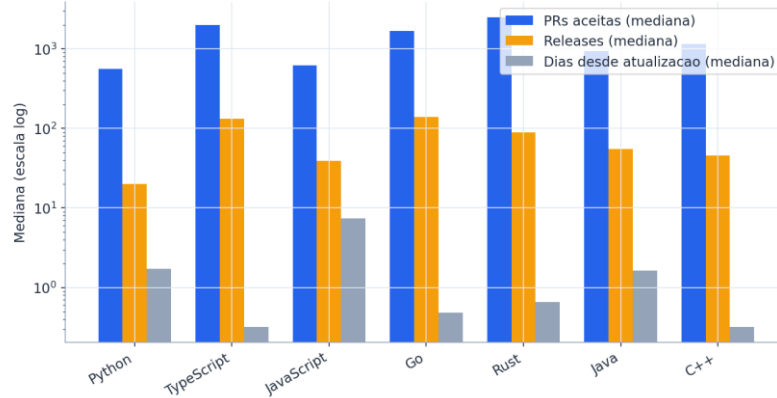


Figura 7 — Mediana de PRs aceitas, releases e dias desde atualização, por linguagem ($n \geq 30$; escala logarítmica).

4.3 Discussão

RQ01 — CONFIRMADA.

A hipótese previa mediana bem acima de cinco anos; o resultado foi 7,75 anos ($Q1 = 3,52$, $Q3 = 11,35$), consistente com a ideia de que acumular estrelas exige tempo de exposição. A cauda de repositórios recém-criados também apareceu como previsto: o mais novo tem apenas 0,01 ano (deepseek-ai/deepseek-harness), mas não deslocou a mediana — confirma que o ciclo recente de IA produz outliers, não uma mudança de padrão.

RQ02 — CONFIRMADA.

Mediana de 768 PRs aceitas, com média de 4.234 — $5,5\times$ a mediana — confirmando a distribuição fortemente assimétrica prevista. O extremo superior (103.316 PRs em firstcontributions/first-contributions) é consistente com a hipótese de que a métrica mede mais modelo de governança do que popularidade pura.

RQ03 — PARCIALMENTE CONFIRMADA.

A fração alta de repositórios sem nenhuma release (28,6%) confirma a metade qualitativa da hipótese — parte relevante dos repositórios mais estrelados não é software versionado. Porém a mediana observada (39 releases) é mais alta do que "baixa" sugeriria, e o quartil superior (146,25) mostra um grupo considerável de projetos com publicação ativa. A censura de 21 repositórios no teto de 1.000 (seção 3.1) não afeta essa mediana, mas reforça que o valor real de contribuição por release é ainda maior do que o medido.

RQ04 — CONFIRMADA (a mais fortemente confirmada das sete, como previsto).

Mediana de apenas 1,81 dias desde a última atualização, exatamente na direção prevista. A cauda longa ($P95 = 748$ dias) mostra que o abandono existe, mas é raro no topo da lista de estrelas — como hipotetizado, um projeto abandonado tende a perder relevância antes de acumular popularidade suficiente para entrar na amostra.

RQ05 — CONFIRMADA.

Python (228), TypeScript (174) e JavaScript (111) dominam o topo — as mesmas três linguagens do topo do Octoverse 2025, mas em ordem diferente (Octoverse: TypeScript, Python, JavaScript), confirmando a divergência esperada entre uma métrica por contribuidores e uma por estrelas. A fração sem linguagem primária (8,7%, maior até que Go, a 5ª posição do ranking) confirma a hipótese de que parte relevante do que é popular no GitHub não é código — são listas de links e material de estudo.

RQ06 — CONFIRMADA.

Mediana de 0,8761 entre os 957 repositórios com pelo menos uma issue, acima do limiar de 0,8 previsto. A ressalva planejada se confirmou necessária: incluir os 43 repositórios sem nenhuma issue reduziria a mediana para 0,8648, uma diferença pequena, mas que justifica reportar separadamente, como o grupo decidiu na metodologia.

RQ07 (bônus) — PARCIALMENTE REFUTADA, com um resultado mais interessante que a hipótese original.

A hipótese previa que a linguagem explicaria pouco. Na prática, há variação considerável entre linguagens: a mediana de PRs aceitas varia de 559,5 (Python) a 2.495 (Rust) — mais de 4x; releases vão de 20 (Python) a 139 (Go); e a atualidade varia de 0,32 dia (TypeScript e C++) a 15,3 dias no grupo "Outras". Rust chama atenção por liderar em PRs aceitas mesmo não estando entre as linguagens mais populares do Octoverse (`is_popular_language = falso`) — popularidade por contribuidores e intensidade de contribuição por repositório não são a mesma coisa. Por outro lado, o grupo "Outras" (272 repositórios, linguagens com $n < 30$) é sistematicamente o mais lento a atualizar e o único com mediana de releases igual a zero — o que sustenta a intuição original do grupo de que o fator relevante pode ser o tipo de projeto (linguagens raras concentram listas, materiais de estudo e ferramentas de nicho) mais do que a linguagem isoladamente. A hipótese de que o padrão apareceria mais em releases do que em PRs também não se confirmou: a variação relativa foi da mesma ordem de grandeza nas três métricas.

Ameaças à validade.

- A amostra é um retrato de um único instante (2026-08-18); repositórios sobem e descem no ranking de estrelas ao longo do tempo, e os resultados não devem ser generalizados como propriedade permanente desses projetos.
- A contagem de releases é subestimada para 21 repositórios (censura no teto de 1.000), o que pode achatar artificialmente a cauda superior da RQ03 e da RQ07 para linguagens dominadas por projetos muito ativos.
- A classificação de linguagem primária vem do GitHub Linguist e reflete o volume de bytes por linguagem no repositório, não necessariamente a linguagem "principal" do ponto de vista do mantenedor — um projeto documentado majoritariamente em Markdown mas com lógica em Python pode aparecer como Undefined ou Markdown.
- O agrupamento "Outras" da RQ07 mistura $44 - 7 = 37$ linguagens com características muito diferentes entre si (de Ruby a Assembly); a mediana desse grupo deve ser lida como uma média de heterogeneidade, não como propriedade de uma linguagem específica.

Contribuição das inovações.

A RQ07 aprofundou a leitura da RQ05: mostrou que a divergência entre a ordem de popularidade por estrelas e por contribuidores (RQ05) se reflete também em como cada linguagem contribui, versiona e atualiza — Rust é um contraexemplo direto de que popularidade por contribuidores (Octoverse) e intensidade de contribuição por repositório não caminham juntas. A validação cruzada via REST não alterou nenhuma conclusão, mas eliminou a possibilidade de que os resultados fossem artefato de erro de mapeamento na consulta GraphQL — o que dá mais peso às confirmações relatadas acima. A auditoria automática do board não afeta os resultados das RQs, mas sustenta a integridade do processo que a seção 3 descreve.

5. Conclusão

Das sete questões de pesquisa investigadas, seis tiveram a hipótese informal confirmada (RQ01, RQ02, RQ04, RQ05, RQ06, e parcialmente RQ03) e uma foi parcialmente refutada de forma produtiva (RQ07): os repositórios mais estrelados do GitHub são, em mediana, projetos maduros (7,75 anos), muito atualizados (1,81 dias desde o último push) e com alta taxa de resolução de issues (0,88), mas nem sempre lançam releases formais — quase 3 em cada 10 nunca publicaram uma — e a linguagem de programação, isoladamente, explica menos sobre o comportamento do projeto do que sua popularidade relativa por contribuidores versus por estrelas.

A principal limitação do estudo é temporal: a amostra reflete um único instante de coleta, e métricas como estrelas e data de último push mudam continuamente. A censura de contagem de releases no teto de 1.000 da API GraphQL também limita a precisão da RQ03 e da RQ07 para a cauda de projetos mais ativos, ainda que não afete as medianas reportadas.

Com mais tempo, o grupo aprofundaria a RQ07 com um teste estatístico formal (ex.: Kruskal-Wallis) para confirmar se as diferenças de mediana entre linguagens são estatisticamente significativas e não apenas descritivas, e investigaria o grupo "Outras" com uma segunda variável — tipo de conteúdo do repositório (lista, documentação, software) — para separar o efeito de linguagem do efeito de tipo de projeto que a discussão da RQ07 apenas sugeriu.

Referências

- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.
- GITHUB. Octoverse 2025. Disponível em: <https://octoverse.github.com/>. Acesso em: 13 ago. 2026.
- GITHUB. GraphQL API v4 documentation. Disponível em: <https://docs.github.com/en/graphql>.